



Institut Teknologi Bandung

Direktorat Pendidikan Profesional Berkelanjutan

×

CHAPTER 5

Dasar-Dasar Machine Learning

Ketegangan antara optimalisasi dan generalisasi – mengapa ia tak terhindarkan, bagaimana mengukurnya, dan apa saja yang benar-benar menolong.

Durasi 3 jam (2 sesi)

Pengajar Prof. Bambang Riyanto Trilaksono

Sumber Chollet & Watson, Deep Learning with Python 3e – bab 5 (hlm. 136-170)

Tujuan Sesi

Setelah sesi ini, peserta dapat:

- 1 Merumuskan **ketegangan optimalisasi vs generalisasi**, dan menyebut tiga penyebab overfitting: data berderau, fitur ambigu, dan fitur langka.
- 2 Menjelaskan **hipotesis manifold** dan mengapa generalisasi deep learning adalah *interpolasi*, bukan penalaran.
- 3 Memilih protokol evaluasi yang tepat – **hold-out, K-lipat, K-lipat berulang** – dan menghindari tiga jebakannya.
- 4 Menetapkan **tolak banding akal sehat** sebelum melatih apa pun.
- 5 Mendiagnosis tiga kegagalan pelatihan: **tidak mulai, tidak menggeneralisasi, tidak bisa overfit** – dan tahu obat masing-masing.
- 6 Menerapkan **kurasi data, rekayasa fitur, early stopping, pengurangan kapasitas, regularisasi bobot, dan dropout**, serta tahu kapan masing-masing yang tepat.

BUKU

[Bab 5 — teks penuh](#)

NOTEBOOK

[01_korelasi_semu_dan_derau.ipynb](#)

NOTEBOOK

[02_label_diacak_manifold.ipynb](#)

NOTEBOOK

[03_protokol_evaluasi.ipynb](#)

NOTEBOOK

[04_kapasitas_model.ipynb](#)

NOTEBOOK

[05_regularisasi_l2_dropout.ipynb](#)

REPO

[Notebook resmi penulis](#)

Dari 'modelnya jalan' ke 'modelnya boleh dipakai'

Bab 4 memperkenalkan overfitting sebagai kejadian. Bab 5 menjadikannya **kerangka berpikir**, dan hampir semua praktik baik di sisa buku ini adalah cara menangani satu ketegangan yang sama.

5.1 · Generalisasi

Apa yang menyebabkan overfitting, dan mengapa deep learning bisa menggeneralisasi sama sekali.

5.2 · Menilai model

Tiga protokol, tolak banding akal sehat, dan tiga jejak yang membatalkan penilaian.

5.3 · Memperbaiki fit

Tiga kegagalan pelatihan dan obatnya. Sasarannya: **bisa** overfit dulu.

5.4 · Memperbaiki generalisasi

Kurasi data, rekayasa fitur, early stopping, dan tiga cara regularisasi.

*Persoalan pokok dalam machine learning adalah ketegangan antara **optimalisasi** dan **generalisasi**. Tujuannya jelas generalisasi yang baik – tetapi Anda tidak mengendalikan generalisasi; yang bisa Anda lakukan hanya memasang model ke data latihnya.*

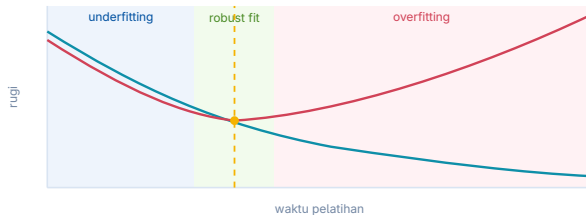
Chollet & Watson, bab 5.1

01

Generalisasi: tujuan machine learning

Mengapa overfitting terjadi di setiap persoalan, tanpa kecuali.

Kurva kanonis – dan ia berlaku universal



Gambar 5.1 — pola ini muncul pada SETIAP jenis model dan SETIAP kumpulan data

Gambar 5.1 – di awal, optimalisasi dan generalisasi berjalan searah. Setelah sekian iterasi, keduanya berpisah.

- ♦ **Underfit** – masih ada kemajuan yang bisa diraih; model belum menangkap semua pola yang relevan.
- ♦ **Robust fit** – titik terbaik. Sempit, dan hanya terlihat lewat metrik validasi.
- ♦ **Overfit** – model mulai mempelajari pola yang **khass data latih** tetapi menyesatkan pada data baru.

Tiga penyebab overfitting

1 · Data berderau

Masukan tak sah (citra MNIST hitam seluruhnya), dan yang lebih buruk: masukan sah yang **salah label**. Model yang memaksa mencakup pencilan ini akan salah pada data mirip.

2 · Fitur ambigu

Data bersih pun bisa berderau bila persoalannya memang tak pasti. Pisang *mentah* vs *matang* tak punya batas objektif; tekanan udara yang sama kadang diikuti hujan, kadang tidak.

3 · Fitur langka & korelasi semu

Kata *cherimoya* muncul sekali, kebetulan di ulasan negatif → model memberinya bobot besar. **Tidak perlu langka-langka amat**: kata yang muncul 100 kali dengan 54% positif sudah cukup.

Selisih 54% lawan 46% itu bisa saja **kebetulan statistik murni**. Model tetap akan memakainya. Chollet menyebut ini **salah satu sumber overfitting yang paling umum**.

Percobaan: tambahkan derau murni, akurasi turun

listing 5.1 – dua kumpulan pembanding

PYTHON

```
(train_images, train_labels), _ = mnist.load_data()
train_images = train_images.reshape((60000, 28 * 28)).astype("float32") / 255

# 784 kanal derau putih ditempelkan - separuh data kini derau
train_images_with_noise_channels = np.concatenate(
    [train_images, np.random.random((len(train_images), 784))], axis=1)

# pembanding: 784 kanal nol, sama-sama tidak informatif
train_images_with_zeros_channels = np.concatenate(
    [train_images, np.zeros((len(train_images), 784))], axis=1)
```

Kandungan informasinya **identik** pada kedua kumpulan. Manusia tidak akan terpengaruh sama sekali oleh penambahan ini.

Tetapi akurasi validasi model yang dilatih dengan kanal derau berakhir **sekitar satu poin persen lebih rendah** – murni akibat korelasi semu. Makin banyak kanal derau, makin jauh merosotnya.

—1 poin

akurasi validasi, hanya karena derau ditempelkan

Obatnya: **seleksi fitur**. Hitung skor kegunaan tiap fitur – misalnya *mutual*

information antara fitur dan label – lalu simpan yang di atas ambang. Membatasi IMDB ke 10.000 kata tersering di bab 4 adalah bentuk kasarnya.

Model bisa memasang diri ke APA SAJA

listing 5.4 – label diacak

PYTHON

```
random_train_labels = train_labels[:]      # salin
np.random.shuffle(random_train_labels)      # putus semua hubungan masukan-target

model = keras.Sequential([
    layers.Dense(512, activation="relu"),
    layers.Dense(10, activation="softmax"),
])
model.compile(optimizer="rmsprop", loss="sparse_categorical_crossentropy",
              metrics=["accuracy"])
model.fit(train_images, random_train_labels,
          epochs=100, batch_size=128, validation_split=0.2)
```

OUTPUT

```
rugi latih : turun terus, mulus
akurasi validasi: bertahan di ~10% (= tolok banding acak untuk 10 kelas)
```

Rugi latih tetap turun walau **tidak ada hubungan apa pun** antara masukan dan label. Model hanya **menghafal, seperti kamus Python**. Jadi: kemampuan memasang diri bukan bukti bahwa persoalannya bisa diselesaikan.

Kalau begitu, mengapa deep learning menggeneralisasi sama sekali? Jawabannya, kata Chollet, **sedikit sekali berkaitan dengan modelnya**, dan banyak berkaitan dengan struktur informasi di dunia nyata.

Hipotesis manifold



256^{784} larik mungkin — jauh lebih banyak dari jumlah atom di alam semesta.
Yang benar-benar berupa tulisan tangan hanya menempati sepetak kecil, dan petak itu bersinambung.

Gambar 5.7 - 5.8 – angka tulisan tangan membentuk manifold di dalam ruang semua larik 28×28 yang mungkin.

Hipotesis manifold menyatakan bahwa semua data alami terletak pada manifold berdimensi rendah di dalam ruang berdimensi tinggi tempat ia disandikan.

Chollet & Watson, bab 5.1.2

- ♦ **Manifold** = subruang berdimensi lebih rendah yang secara setempat menyerupai ruang linear. Kurva mulus di bidang adalah manifold 1D dalam ruang 2D.
- ♦ Ia **bersinambung**: ubah satu sampel sedikit, ia tetap angka yang sama. Dua angka mana pun dihubungkan oleh **lintasan mulus**.
- ♦ Berlaku untuk MNIST, wajah manusia, bentuk pohon, suara manusia, bahkan bahasa alami.

Interpolasi – dan batasnya

Yang bisa dilakukan model

- ♦ Memahami titik yang belum pernah dilihat dengan **men-gaitkannya ke titik terdekat** di manifold.
- ♦ Memahami keseluruhan ruang hanya dari sebagian sampelnya – *mengisi bagian yang kosong*.
- ♦ Chollet menyebutnya **generalisasi setempat**.

Yang tidak bisa

- ♦ **Generalisasi ekstrem** – yang manusia lakukan setiap hari.
- ♦ Anda bisa berpindah seminggu di NYC, seminggu di Shanghai, seminggu di Bangalore **tanpa ribuan kali se-umur hidup berlatih untuk tiap kota**.
- ♦ Itu ditopang abstraksi, model simbolik, penalaran, logika, akal sehat – yang kita sebut **nalar**, bukan intuisi.

Peringatan Chollet: interpolasi hanya **puncak gunung es**. Menganggap interpolasi sama dengan seluruh generalisasi adalah kekeliruan. Bab 19 kembali ke sini.

Perhatikan juga: interpolasi **pada manifold laten** berbeda dari interpolasi linear di ruang induk. Rerata piksel dua angka MNIST biasanya **bukan angka yang sah**.

Mengapa deep learning bekerja – dan syaratnya

- 1 Kurva itu punya cukup parameter untuk memasangkan diri ke **apa saja** – kalau dibiarkan cukup lama, ia murni menghafal.
- 2 Tetapi data yang dipasangkan **bukan titik-titik terpengar**; ia membentuk manifold berdimensi rendah yang sangat terstruktur.
- 3 Karena pemasangannya berlangsung **bertahap dan mulus**, ada titik di tengah pelatihan saat kurva model kira-kira menghampiri manifold alami data itu (gambar 5.10).

Sifat 1 · pemetaan mulus dan bersinambung

Wajib, karena harus terdiferensialkan. Kemulusan itu justru yang membantu menghampiri manifold laten.

Sifat 2 · prior arsitektur

Strukturnya mencerminkan 'bentuk' informasi pada datanya – terutama pada model citra (bab 8-12) dan model deret (bab 13).

Data latih adalah yang terpenting

Satu-satunya yang akan Anda temukan di dalam model deep learning adalah apa yang Anda masukkan ke dalamnya: prior yang tersandi di arsitekturnya, dan data yang dipakai melatihnya.

Chollet & Watson, bab 5.1.3

- ♦ Kemampuan menggeneralisasi lebih merupakan akibat **struktur alami data** ketimbang sifat model.
- ♦ Deep learning adalah pemasangan kurva, jadi ia butuh **pencuplikan yang padat** atas ruang masukannya – terutama di dekat batas keputusan (gambar 5.11).
- ♦ Cuplikan jarang → kurva yang dipelajari tidak cocok dengan ruang laten, dan **interpolasinya salah**.
- ♦ Karena itu: **cara terbaik memperbaiki model adalah melatihnya dengan lebih banyak data, atau data yang lebih baik**.

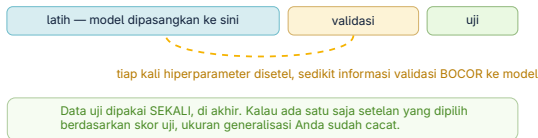
Kalau menambah data tidak mungkin, pilihan terbaik berikutnya adalah **membatasi banyaknya informasi yang boleh disimpan model**, atau menambah kekangan pada kemulusan kurvanya. Itulah **regularisasi**, dan itu isi bagian 5.4.4.

02

Menilai model machine learning

Anda hanya bisa mengendalikan yang bisa Anda amati.

Mengapa dua himpunan tidak cukup



Menyetel hiperparameter berdasarkan skor validasi adalah bentuk pembelajaran juga – dan karenanya bisa overfit ke himpunan validasi.

- ♦ **Hiperparameter** (jumlah lapis, ukuran lapis) berbeda dari **parameter** (bobot). Yang pertama Anda setel; yang kedua dipelajari.
- ♦ Tiap kali Anda menyetel satu hiperparameter berdasarkan skor validasi, **sedikit informasi tentang data validasi bocor ke model**.
- ♦ Sekali dua kali tidak apa-apa. Diulang berkali-kali, himpunan validasi berhenti menjadi ukuran yang jujur.
- ♦ Karena itu perlu himpunan ketiga: **data uji**, yang modelnya belum pernah menyentuhnya, bahkan tidak langsung.

Tiga protokol evaluasi

PROTOKOL	CARA KERJANYA	KAPAN DIPAKAI	BIAYANYA
Hold-out sederhana	Sisihkan sebagian sebagai uji; sisanya dibagi latih dan validasi.	Data banyak.	1 model
K-lipat	Bagi jadi K bagian sama besar; tiap bagian sekali jadi validasi; skornya dirata-ratakan.	Skor sangat berayun bergantung belahan.	K model
K-lipat berulang, diacak	Jalankan K-lipat P kali, acak ulang tiap kali.	Data sedikit dan penilaian harus setepat mungkin.	$P \times K$ model

Listing 5.6 – Inti K-lipat

PYTHON

```
k = 3
num_validation_samples = len(data) // k
np.random.shuffle(data)
validation_scores = []
for fold in range(k):
    validation_data = data[num_validation_samples * fold :
                           num_validation_samples * (fold + 1)]
    training_data = np.concatenate(
        [data[: num_validation_samples * fold],
         data[num_validation_samples * (fold + 1) :]])
    model = get_model() # instans BARU, belum terlatih, tiap lipatan
    model.fit(training_data, ...)
    validation_scores.append(model.evaluate(validation_data, ...))

validation_score = np.average(validation_scores)
model = get_model()
model.fit(data, ...) # model akhir: dilatih di SEMUA data non-uji
test_score = model.evaluate(test_data, ...)
```

Perhatikan `model = get_model()` di dalam gelung: tiap lipatan wajib memakai **instans yang benar-benar baru**. Memakai ulang model yang sudah terlatih membatalkan seluruh penilaian.

Tolok banding akal sehat: altimeter roket yang tak terlihat

Melatih model deep learning itu seperti menekan tombol yang meluncurkan roket di dunia paralel. Anda tak bisa mendengarnya, tak bisa melihatnya. Satu-satunya umpan balik yang Anda punya adalah metrik validasi – seperti altimeter pada roket yang tak terlihat itu.

Chollet & Watson, bab 5.2.2

PERSOALAN	TOLOK BANDING AKAL SEHAT	ALASANNYA
MNIST	> 0,10	Penebak acak atas 10 kelas seimbang.
IMDB	> 0,50	Dua kelas, seimbang 50:50.
Reuters	≈ 0,18-0,19	46 kelas, tetapi sebarannya tidak rata.
Biner 90:10	> 0,90	Penebak yang selalu menjawab kelas A sudah dapat 0,90. Anda harus melampauinya.

Kalau Anda **tidak bisa mengalahkan penyelesaian sepele**, model Anda tidak berharga. Mungkin modelnya salah, atau – dan ini yang sering – **persoalannya memang tidak bisa didekati dengan machine learning**. Kembali ke papan gambar.

Tiga jebakan yang membatalkan penilaian

1 · Keterwakilan data

Data terurut menurut kelas, lalu 80% pertama diambil sebagai latih → latih hanya berisi kelas 0-7, uji hanya 8-9. *Kelihatannya konyol, tetapi mengejutkan seringnya. Obatnya: acak sebelum membelah.*

2 · Anak panah waktu

Meramal masa depan dari masa lalu? **Jangan diacak.** Mengacak menciptakan **kebocoran waktu**: model dilatih dengan data dari masa depan. Semua data uji harus lebih baru dari data latih.

3 · Data kembar

Titik data yang muncul dua kali – lazim pada data dunia nyata. Setelah diacak, salinannya tersebar ke latih dan validasi: Anda menguji di atas data latih sendiri. **Yang terburuk dari semuanya.**

Pada data transaksional, jebakan **2 dan 3** hampir selalu muncul bersamaan: deranya bersifat temporal, dan subjek yang sama muncul berkali-kali. Membelah secara acak per baris **melanggar keduanya** dalam satu langkah.

03

Memperbaiki fit

Untuk mencapai fit yang pas, Anda harus overfit dulu.

Tiga kegagalan, tiga obat

Untuk mencapai fit yang sempurna, Anda harus overfit lebih dulu. Karena Anda tidak tahu di mana batasnya, Anda harus melewatinya untuk menemukannya.

Chollet & Watson, bab 5.3

GEJALANYA	ARTINYA	YANG DICoba
Pelatihan tidak mulai – rugi latih tidak turun	Setelan penurunan gradien salah.	Setel learning rate (turunkan atau naikan) dan ukuran batch . Yang lain dibiarkan tetap.
Mulai, tetapi tidak menggeneralisasi – tolok banding tak terkalahkan	Ada yang salah secara mendasar.	Mungkin datanya tidak memuat informasi untuk meramal targetnya; atau prior arsitekturnya salah untuk jenis data ini.
Menggeneralisasi, tetapi tidak bisa overfit	Kapasitas kurang.	Tambah lapis, perbesar lapis, atau pakai jenis lapis yang lebih cocok .

Kegagalan kedua adalah **situasi terburuk** dalam machine learning: ia menandakan ada yang salah secara mendasar, dan **sering tidak mudah dikenali apanya**.

Learning rate: satu angka yang menghentikan segalanya

listing 5.7 - terlalu besar

PYTHON

```
model.compile(
    optimizer=keras.optimizers.RMSprop(
        learning_rate=1.0),
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"])
model.fit(train_images, train_labels,
        epochs=10, batch_size=128,
        validation_split=0.2)
```

OUTPUT

akurasi mentok di 20%-40%
dan tidak mau lewat dari situ

listing 5.8 - wajar

PYTHON

```
model.compile(
    optimizer=keras.optimizers.RMSprop(
        learning_rate=1e-2),
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"])
model.fit(train_images, train_labels,
        epochs=10, batch_size=128,
        validation_split=0.2)
```

OUTPUT

model sekarang bisa dilatih

- Learning rate **terlalu tinggi** → pembaruan melampaui jauh titik yang pas; hasilnya seperti di kiri.
- **Terlalu rendah** → pelatihan begitu lambat sampai **tampak macet**, padahal sebetulnya jalan.
- **Perbesar ukuran batch** → gradien lebih informatif dan kurang berisik (ragamnya lebih rendah).

Semua parameter ini saling bergantung. Chollet menyarankan: cukup setel **learning rate dan ukuran batch saja**, sisanya biarkan tetap.

Kapasitas: terlalu kecil, pas, terlalu besar

MODEL	ARSITEKTUR	YANG TERJADI	GAMBAR
Kapasitas kurang	<code>Dense(10, softmax)</code> saja – regresi logistik	Rugi validasi turun ke 0,26 lalu diam di situ . Bisa fit, tetapi tidak bisa jelas-jelas overfit .	5.14
Kapasitas pas	2 × <code>Dense(128, relu)</code>	Fit cepat, mulai overfit setelah 8 epoch . Persis seperti seharusnya.	5.15
Kapasitas berlebih	3 × <code>Dense(2048, relu)</code>	Overfit langsung sejak awal ; rugi latih hampir nol dengan cepat, rugi validasi berisik.	5.16

listing 5.9 – kapasitas kurang

PYTHON

```
model = keras.Sequential([layers.Dense(10, activation="softmax")])
model.compile(optimizer="rmsprop", loss="sparse_categorical_crossentropy",
              metrics=["accuracy"])
history_small_model = model.fit(train_images, train_labels,
                                epochs=20, batch_size=128, validation_split=0.2)

# catatan: saat melatih model besar, kecilkan batch_size agar memori tidak jebol
# (batch_size=32 pada versi 3 x 2048 unit)
```

Alur kerjanya: **mulai dari sedikit lapis dan parameter, lalu besarkan sampai rugi validasi berhenti membaik**. Tidak ada rumus ajaib untuk menebak ukuran yang benar – **harus dicoba, di himpunan validasi, bukan di himpunan uji**.

04

Memperbaiki generalisasi

Lima cara, berurut dari yang paling berdampak.

Kurasi data – imbal hasil terbesar, dan sering dilewati

Deep learning adalah pemasangan kurva, bukan sihir.

Chollet & Watson, bab 5.4.1

Mengeluarkan lebih banyak tenaga dan uang untuk **pengumpulan data** hampir selalu memberi imbal hasil **jauh lebih besar** daripada jumlah yang sama untuk mengembangkan model yang lebih baik.

- ➊ **Pastikan datanya cukup.** Anda butuh pencuplikan yang padat atas ruang masukan-silang-keluaran. Persoalan yang tampak mustahil kadang jadi bisa diselesaikan begitu datanya lebih banyak.
- ➋ **Kecilkan galat pelabelan.** Lihat sendiri masukannya untuk mencari kejanggalan; periksa ulang labelnya.
- ➌ **Bersihkan data dan tangani nilai yang hilang.** (Bab 6 membahasnya.)
- ➍ **Lakukan seleksi fitur** kalau fiturnya banyak dan Anda belum yakin mana yang berguna.

Batasnya jelas: kalau persoalannya **terlalu berderau atau pada dasarnya diskret** – misalnya mengurutkan senarai – deep learning **tidak akan menolong**, seberapa pun datanya.

Rekayasa fitur: contoh jam dinding

TINGKAT	MASUKANNYA	YANG DIBUTUHKAN
Data mentah	Kisi piksel citra jam	ConvNet, dan sumber daya komputasi yang tidak sedikit.
Fitur lebih baik	Koordinat (x, y) ujung tiap jarum	Algoritma machine learning sederhana sudah cukup.
Lebih baik lagi	Sudut θ tiap jarum (koordinat polar)	Tidak perlu machine learning sama sekali – cukup pembulatan dan pencarian di kamus.

Itulah inti rekayasa fitur: **membuat persoalan lebih mudah dengan menyatakannya secara lebih sederhana**. Membuat manifold latennya lebih mulus, lebih sederhana, lebih rapi. Ia menuntut pemahaman mendalam atas persoalannya.

Tetap berguna — alasan 1

Fitur yang baik menyelesaikan persoalan **dengan sumber daya lebih sedikit**. Memakai ConvNet untuk membaca jam dinding itu konyol.

Tetap berguna — alasan 2

Fitur yang baik menyelesaikan persoalan dengan **data jauh lebih sedikit**. Kemampuan model menemukan fiturnya sendiri bergantung pada tersedianya banyak data latih.

Sebelum deep learning, rekayasa fitur adalah **bagian terpenting** alur kerja machine learning. Solusi MNIST dulu dibangun dari fitur yang ditulis tangan: jumlah gelung pada citra angka, tinggi tiap angka, histogram nilai piksel.

Early stopping

Cara di bab 4 – latih lebih lama dari perlu untuk menemukan epoch terbaik, lalu latih model baru tepat sebanyak epoch itu.

Lazim, tetapi menuntut **pekerjaan berulang** yang kadang mahal.

Cara yang lebih baik – simpan model tiap akhir epoch, lalu pakai yang terbaik. Di Keras ini dikerjakan callback `EarlyStopping`, yang menghentikan pelatihan begitu metrik validasi berhenti membaik **sambil mengingat keadaan model terbaik**.

Callback dibahas penuh di **bab 7**.

batas antara kurva underfit dan overfit – adalah **salah satu hal paling ampuh** yang bisa Anda lakukan untuk memperbaiki generalisasi.

Menemukan titik persis saat fit paling bisa digeneralisasi –

Regularisasi 1: kecilkan modelnya

VERSI	LAPIS ANTARA	MULAI OVERFIT	PERILAKUNYA	GAMBAR
Acuan	2 × Dense(16)	epoch 4	Titik banding.	—
Lebih kecil	2 × Dense(4)	epoch 6	Mulai overfit lebih lambat, dan memburuknya lebih pelan.	5.18
Jauh lebih besar	2 × Dense(512)	epoch 1	Overfit hampir seketika dan jauh lebih parah; rugi validasinya juga lebih berisik.	5.19

Tidak ada rumus ajaib untuk ukuran yang benar. Alur kerjanya: **mulai dari sedikit, besarkan sampai imbal hasil rugi validasi mengecil** – dan lakukan itu di **himpunan validasi, tentu saja, bukan himpunan uji.**

Regularisasi 2: kekang bobotnya (L1 / L2)

Pisau cukur Occam: diberi dua penjelasan atas suatu hal, yang paling mungkin benar adalah yang paling sederhana – yang paling sedikit andaianya.

Prinsip yang dipakai Chollet untuk membenarkan regularisasi bobot

listing 5.13-5.14

PYTHON

```
from keras.regularizers import l2
from keras import regularizers

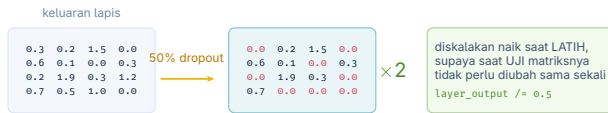
model = keras.Sequential([
    layers.Dense(16, kernel_regularizer=l2(0.002), activation="relu"),
    layers.Dense(16, kernel_regularizer=l2(0.002), activation="relu"),
    layers.Dense(1, activation="sigmoid"),
])

# pilihan lain:
regularizers.l1(0.001)           # l1
regularizers.l1_l2(l1=0.001, l2=0.001)  # l1 dan l2 sekaligus
```

JENIS	BIAYA YANG DITAMBAHKAN KE RUGI	EFEKNYA
L1	Sebanding dengan nilai mutlak koefisien bobot (norma L1).	Mendorong bobot menjadi jarang (banyak yang nol).
L2	Sebanding dengan kuadrat koefisien bobot (norma L2).	Mendorong semua bobot menjadi kecil. Disebut juga weight decay – namanya beda, matematikanya sama.

`l2(0.002)` berarti tiap koefisien menambahkan `0.002 * nilai_bobot ** 2` ke rugi total. Denda ini hanya ditambahkan **saat latih**, jadi **rugi saat latih akan tampak jauh lebih tinggi daripada saat uji** – dan itu normal, bukan bug.

Regularisasi 3: dropout



Gambar 5.21 — laju dropout lazimnya 0,2 sampai 0,5. Saat uji tidak ada unit yang dijatuhkan.

Gambar 5.21 – dropout pada matriks aktivasi saat latih, dengan penskalaan dikerjakan saat latih juga.

listing 5.15 – dropout pada model IMDB

PYTHON

```
model = keras.Sequential([
    layers.Dense(16, activation="relu"),
    layers.Dropout(0.5),
    layers.Dense(16, activation="relu"),
    layers.Dropout(0.5),
    layers.Dense(1, activation="sigmoid"),
])
```

Dari mana ide dropout datang – kisah Hinton dan bank

Saya pergi ke bank saya. Tellernya berganti terus dan saya tanya salah satunya kenapa. Dia bilang tidak tahu, tetapi mereka memang sering dipindah-pindah. Saya kira itu pasti karena menipu bank memerlukan kerja sama antar-pegawai. Dari situ saya sadar bahwa membuang subhimpunan neuron yang berbeda secara acak pada tiap contoh akan mencegah persekongkolan, dan dengan begitu mengurangi overfitting.

Geoff Hinton, dikutip Chollet & Watson, bab 5.4.4

Gagasan intinya: menyuntikkan derau ke nilai keluaran sebuah lapis **memutus pola kebetulan yang tidak bermakna** – yang Hinton sebut *persekongkolan* – yang akan mulai dihafal model bila tidak ada derau sama sekali.

Asal-usulnya kebetulan menolong sebagai analogi: **rotasi pegawai sebagai kendali kecurangan** adalah kendali internal yang sudah dikenal luas. Dropout adalah kendali yang sama, diterapkan pada neuron.

Hasilnya pada model IMDB (gambar 5.22): peningkatan yang jelas atas model acuan, dan **tampak bekerja lebih baik daripada regularisasi L2**, sebab rugi validasi terendah yang dicapainya lebih rendah.

Mana yang dipakai kapan

CARA	KAPAN PALING COCOK	CATATAN
Data lebih banyak / lebih baik	Selalu, kalau mungkin.	Imbal hasil terbesar. Menambah data yang terlalu berderau justru merugikan.
Fitur yang lebih baik	Data sedikit; persoalan yang dipahami mendalam.	Bisa menghapus kebutuhan akan model besar sama sekali.
Kurangi kapasitas	Model kecil; data sedikit.	Cari kompromi – jangan sampai malah underfit.
Regularisasi bobot (L1/L2)	Model deep learning yang lebih kecil.	Pada model besar yang sangat berparameter lebih, mengekang nilai bobot tidak banyak berpengaruh .
Dropout	Model besar – di situ regularisasi bobot kurang mempan.	Laju lazim 0,2-0,5. Pada IMDB hasilnya mengalahkan L2.

Satu syarat menaungi semuanya: **regularisasi harus selalu dipandu prosedur evaluasi yang tepat**. Anda hanya akan mencapai generalisasi **kalau Anda bisa mengukurnya**.

Yang wajib terbawa dari bab 5

- 1 Tujuan model adalah **menggeneralisasi**: tepat pada masukan yang belum pernah dilihat. Lebih sulit daripada kelihatannya.
- 2 Jaringan dalam menggeneralisasi dengan **menginterpolasi** pada manifold laten data. Karena itu ia hanya memahami masukan yang **dekat dengan yang pernah dilihatnya**.
- 3 Persoalan pokoknya adalah **ketegangan optimalisasi vs generalisasi**. Setiap praktik baik di buku ini menangani ketegangan itu.
- 4 **Ukur dulu, baru perbaiki**. Hold-out, K-lipat, K-lipat berulang – dan selalu sisakan himpunan uji yang benar-benar tak tersentuh.
- 5 **Tetapkan tolok banding akal sehat** sebelum melatih. Kalau tak terkalahkan, mungkin persoalannya bukan persoalan machine learning.
- 6 Untuk fit: setel **learning rate dan ukuran batch**, perbaiki **prior arsitektur**, tambah **kapasitas** sampai bisa overfit.
- 7 Untuk generalisasi: **data lebih baik** → **fitur lebih baik** → **early stopping** → **kurangi kapasitas** → **L1/L2** → **dropout**, dalam urutan itu.

NOTEBOOK

[02_label_diacak_manifold.ipynb](#)

NOTEBOOK

[05_regularisasi_l2_dropout.ipynb](#)

BAB BERIKUT

[Bab 6 — Alur kerja universal](#)